

Proyecto 8: Monitoreo de clúster HAProxy con Datadog

(Mayo 2026)

Juan Fernando Salazar Serrano, *IEEE*

Abstract—Este proyecto presenta la implementación de una arquitectura de monitoreo y observabilidad para un clúster basado en HAProxy, integrado con Datadog para la supervisión de infraestructura en tiempo real. La solución propuesta incluye un balanceador de carga configurado con múltiples instancias backend desarrolladas en Spring Boot, generación de tráfico mediante Artillery, recolección de métricas a través de Datadog Agent y enriquecimiento del monitoreo mediante endpoints compatibles con Prometheus. La infraestructura fue contenerizada utilizando Docker y orquestada mediante Docker Compose, garantizando portabilidad y reproducibilidad del entorno. Se ejecutaron diferentes escenarios de tráfico, incluyendo carga normal, picos de alta demanda, operaciones CRUD y fallos controlados, con el fin de evaluar el comportamiento del sistema bajo distintas condiciones operacionales. Los resultados evidenciaron coherencia entre el tráfico generado y las métricas observadas, permitiendo visualizar latencia, tasa de solicitudes, disponibilidad de backends y porcentaje de errores a través de dashboards personalizados. Adicionalmente, los mecanismos de alertamiento automatizado y centralización de logs demostraron ser efectivos para la detección de fallos y el análisis operacional. La implementación resalta la importancia de las plataformas de observabilidad en sistemas distribuidos modernos, facilitando el monitoreo proactivo, la respuesta temprana ante incidentes y el fortalecimiento de la confiabilidad de la infraestructura.

I. INTRODUCCIÓN

En los entornos modernos de infraestructura, la disponibilidad,

¹Manuscript received October 9, 2001. (Write the date on which you submitted your paper for review.) This work was supported in part by the U.S. Department of Commerce under Grant BS123456 (sponsor and financial support acknowledgment goes here). Paper titles should be written in uppercase and lowercase letters, not all uppercase. Avoid writing long formulas with subscripts in the title; short formulas that identify the elements are fine (e.g., "Nd-Fe-B"). Do not write "(Invited)" in the title. Full names of authors are preferred in the author field, but are not required. Put a space between authors' initials.

F. A. Author is with the National Institute of Standards and Technology, Boulder, CO 80305 USA (corresponding author to provide phone: 303-555-5555; fax: 303-555-5555; e-mail: author@boulder.nist.gov).

S. B. Author, Jr., was with Rice University, Houston, TX 77005 USA. He is now with the Department of Physics, Colorado State University, Fort Collins, CO 80523 USA (e-mail: author@lamar.colostate.edu).

T. C. Author is with the Electrical Engineering Department, University of Colorado, Boulder, CO 80309 USA, on leave from the National Research Institute for Metals, Tsukuba, Japan (e-mail: author@nrim.go.jp).

el rendimiento y la capacidad de respuesta de los servicios son factores críticos para garantizar una operación confiable. Cuando una aplicación se distribuye en varios backends detrás de un balanceador de carga como HAProxy, la supervisión deja de ser un complemento y se convierte en una necesidad operativa. En este contexto, contar únicamente con el funcionamiento básico del sistema no es suficiente: también es indispensable observar métricas de tráfico, tiempos de respuesta, estado de los servidores, registros de eventos y posibles condiciones de error en tiempo real. Este proyecto aborda la implementación de un sistema integral de monitoreo para un clúster basado en HAProxy, utilizando Datadog como plataforma de observabilidad. A través de métricas, logs y alertas centralizadas, se busca obtener una visión completa del comportamiento del balanceador y de los servicios backend, permitiendo identificar saturación, fallos de disponibilidad, degradación del rendimiento y patrones de carga durante pruebas controladas. Además, la generación de tráfico con Artillery permitirá simular escenarios reales y validar la respuesta del sistema bajo distintas condiciones de uso.

II. CONTEXTO (PROBLEMA)

En muchas arquitecturas distribuidas, el monitoreo se limita a verificar si un servicio “está arriba”, sin profundizar en su comportamiento interno ni en la relación entre carga y desempeño. Esto representa un problema, porque un sistema puede permanecer disponible y aun así operar con alta latencia, errores intermitentes, colas saturadas o balanceo ineficiente entre nodos. Sin una plataforma de observabilidad adecuada, estos síntomas suelen detectarse tarde, cuando ya afectan la experiencia del usuario o la estabilidad del servicio.

En el caso de un clúster con HAProxy y múltiples backends, el reto no solo consiste en enrutar tráfico correctamente, sino también en comprender cómo se comporta cada componente bajo demanda variable. Se necesita una solución que permita recolectar métricas técnicas, centralizar logs estructurados, visualizar tendencias en dashboards y activar alertas ante anomalías. Por ello, el problema que resuelve este proyecto es la falta de visibilidad integral sobre la salud, el rendimiento y la trazabilidad de un clúster computacional, proponiendo una implementación basada en Datadog, contenedores Docker y pruebas de carga con Artillery.

III. ALTERNATIVAS DE SOLUCIÓN

1) Datadog

Qué es: plataforma SaaS de observabilidad unificada para métricas, logs, trazas y alertas.

Ventajas

- Integración muy rápida con HAProxy, Docker y

backends.

- Dashboards y alertas listos para usar.
- Muy buena correlación entre métricas, logs y eventos.
- Menos trabajo operativo: no administras tu propio stack de monitoreo.
- Muy útil para demos y proyectos académicos porque visualmente queda sólido.

Desventajas

- Es de pago y la clave API debe gestionarse con cuidado.
- Dependencia de un tercero externo.
- Menos control fino sobre almacenamiento y retención que en un stack autogestionado.

Cuándo elegirlo: cuando quieres rapidez, buena presentación y mínima fricción operativa.

2) Prometheus + Grafana

Qué es: Prometheus recolecta métricas; Grafana las visualiza. Es la alternativa más común a Datadog en entornos open source.

Ventajas

- Código abierto y muy usado en infraestructura moderna.
- Excelente para métricas time-series.
- Prometheus se integra bien con HAProxy vía OpenMetrics/Exporter.
- Grafana permite dashboards muy flexibles y atractivos.
- Gran ecosistema de exporters y alerting.

Desventajas

- Logs no vienen integrados de forma nativa como en Datadog; normalmente necesitas Loki, Elasticsearch/OpenSearch u otra solución adicional.
- Más componentes que administrar.
- Requiere más configuración manual para alertas, retención y almacenamiento.
- La experiencia “todo en uno” es menor que en Datadog.

Cuándo elegirlo: cuando quieres una solución open source, más controlada y con costo bajo.

3) AWS CloudWatch / AWS Observability

Qué es: monitoreo nativo de AWS para métricas, logs y alertas, con integración directa a la nube.

Ventajas

- Muy buena opción si todo corre en AWS.
- Integración nativa con EC2, ECS, EKS, ALB, RDS, etc.
- No necesitas montar tanto software propio.
- CloudWatch Logs, Metrics y Alarms cubren la base operativa.
- Fácil de operar dentro del ecosistema AWS.

Desventajas

- Menos amigable para observabilidad avanzada que Datadog o Grafana.
- Dashboards y análisis suelen sentirse más limitados.
- Para HAProxy autogestionado fuera de AWS, la integración no es tan natural.
- Puede salir costoso si hay mucho volumen de logs.

Cuándo elegirlo: cuando el proyecto está completamente sobre AWS y quieres centralización nativa.

4) ELK / OpenSearch Stack

Qué es: Elasticsearch/OpenSearch + Logstash/Fluentd + Kibana/OpenSearch Dashboards para logs y visualización.

Ventajas

- Muy fuerte para análisis de logs.
- Buen motor de búsqueda y exploración.
- Puede manejar logs estructurados JSON muy bien.
- Flexible para centralización y correlación de eventos.

Desventajas

- Más pesado de operar.
- No es la opción más cómoda para métricas de infraestructura puras.
- Requiere más mantenimiento y tuning.
- Para una demo o proyecto pequeño, puede ser excesivo.

Cuándo elegirlo: cuando el foco principal es logging y análisis de eventos, no tanto métricas en tiempo real.

5) Zabbix

Qué es: solución clásica de monitoreo de infraestructura.

Ventajas

- Muy sólido para monitoreo de hosts, servicios y alertas.
- Gratis y autohospedado.
- Bueno para infraestructura tradicional.
- Bastante maduro y estable.

Desventajas

- Menos moderno para observabilidad integral.
- Dashboards y experiencia visual suelen ser menos flexibles.
- No es tan natural para métricas tipo Prometheus/OpenMetrics.
- Logs y trazas no son su punto fuerte.

Cuándo elegirlo: cuando buscas monitoreo clásico de infraestructura más que observabilidad avanzada.

Alternativas para el balanceador/proxy

A) HAProxy

Ventajas

- Muy robusto, estable y rápido.
- Excelente para balanceo L7 y L4.
- Buenas estadísticas y logging.
- Muy apropiado para proyectos de observabilidad.

Desventajas

- Configuración más manual.
- Menos “plug and play” que otras opciones modernas.

Cuándo usarlo: cuando el objetivo incluye balanceo serio y observabilidad de tráfico.

B) NGINX

Ventajas

- Muy popular y ampliamente conocido.
- Bueno como reverse proxy y balanceador.
- Configuración relativamente simple.
- Fácil de integrar con logs JSON y monitoreo.

Desventajas

- Menos especializado que HAProxy en balanceo puro y métricas detalladas.
- Algunas capacidades avanzadas dependen de NGINX Plus o configuración adicional.

Cuándo usarlo: cuando quieres una opción más común y sencilla para proxy/reverse proxy.

C) Traefik

Ventajas

- Muy cómodo en Docker y Kubernetes.
- Descubre servicios automáticamente.
- Bueno para entornos dinámicos.
- Integración moderna con observabilidad.

Desventajas

- Menos control fino que HAProxy.
- No siempre es la mejor opción para escenarios de análisis profundo de tráfico.
- Para un proyecto académico de observabilidad de clúster, puede parecer más “orquestador” que balanceador clásico.

Cuándo usarlo: cuando el entorno cambia mucho y quieres descubrimiento automático.

D) Envoy

Ventajas

- Muy potente para observabilidad, métricas y control de tráfico.
- Muy usado en arquitecturas cloud-native y service mesh.
- Excelente para telemetría.

Desventajas

- Curva de aprendizaje alta.
- Más complejo de configurar que HAProxy o NGINX.
- Puede ser demasiado para una práctica universitaria

simple.

Cuándo usarlo: cuando buscas una solución avanzada y moderna de networking.

- **Carga alta o picos (burst traffic):** incrementa significativamente el número de peticiones concurrentes para analizar el desempeño bajo estrés.
- **Tráfico de error:** ejecuta endpoints diseñados para producir respuestas HTTP controladas (400, 404, 503 y timeouts), permitiendo validar la capacidad de monitoreo y alertamiento.

E) AWS ALB / NLB

Ventajas

- Gestión completamente administrada.
- Muy fácil si estás en AWS.
- Métricas y logs integrados con CloudWatch.
- Alta disponibilidad nativa.

Desventajas

- Menor control que un balanceador autogestionado.
- Dependencia fuerte de AWS.
- No te deja mostrar tanto trabajo de configuración como HAProxy.

Cuándo usarlo: cuando el objetivo es infraestructura administrada en AWS, no demostración técnica del proxy.

Estas solicitudes son dirigidas al balanceador HAProxy mediante peticiones HTTP, lo que permite observar en tiempo real cómo la infraestructura responde ante cambios en el patrón de consumo.

2. Balanceador de carga HAProxy

El núcleo de la arquitectura está compuesto por **HAProxy 2.6**, configurado como punto central de entrada para todas las solicitudes entrantes.

Este componente cumple varias responsabilidades críticas:

Distribución de carga (Load Balancing)

HAProxy implementa un algoritmo **Round Robin**, el cual distribuye las solicitudes de manera equitativa entre tres instancias backend de Spring Boot. Esto permite:

- Reducir cuellos de botella.
- Distribuir carga computacional.
- Mejorar disponibilidad.
- Evitar sobrecarga de un único nodo.

Cuando Artillery genera tráfico, HAProxy decide dinámicamente qué backend atenderá cada petición.

Health Checks

El balanceador realiza **verificaciones de salud automáticas** sobre cada backend mediante endpoints de monitoreo como:

/api/health

Estas comprobaciones permiten detectar fallos de disponibilidad. Si un backend deja de responder o devuelve un estado no válido, HAProxy lo marca automáticamente como **DOWN**, retirándolo temporalmente del balanceo hasta recuperar su estado saludable.

Este mecanismo incrementa la tolerancia a fallos y mantiene continuidad operativa del sistema.

Endpoint de métricas Prometheus

IV. DISEÑO DE LA SOLUCIÓN

La solución propuesta se diseñó bajo un enfoque de arquitectura distribuida, observable y contenerizada, cuyo propósito principal consiste en monitorear el comportamiento de un clúster de servicios backend balanceados mediante **HAProxy**, utilizando **Datadog** como plataforma central de observabilidad y **Artillery** como herramienta de generación de tráfico. La arquitectura fue implementada dentro de un entorno **Docker Compose**, permitiendo aislar cada componente funcional y facilitar tanto el despliegue como la reproducibilidad del sistema.

1. Capa de generación de tráfico (Cliente / Load Testing)

En el extremo izquierdo de la arquitectura se encuentra el componente **Artillery**, encargado de simular el comportamiento de clientes reales mediante la generación controlada de solicitudes HTTP hacia el sistema.

El diseño contempla múltiples patrones de tráfico:

- **Tráfico normal (steady RPS):** simula una carga estable y sostenida para observar el comportamiento base del sistema.

HAProxy fue compilado con soporte **PROMEX** (**USE_PROMEX=1**), permitiendo exponer métricas operacionales mediante:

/metrics

Este endpoint entrega métricas relacionadas con:

- Requests por segundo.
- Estado de backends.
- Conexiones activas.
- Latencias.
- Errores HTTP.
- Tiempo de respuesta.

Aunque el sistema usa Datadog como plataforma principal de monitoreo, la incorporación del endpoint Prometheus introduce interoperabilidad y extensibilidad futura con otras herramientas del ecosistema cloud-native.

Estadísticas administrativas

Se habilitó un panel de estadísticas accesible desde:

:8404/stats

El panel permite observar visualmente:

- Backends activos/inactivos.
- Número de conexiones.
- Throughput.
- Estado del balanceador.

Adicionalmente, se implementó un **Unix Socket compartido**, usado por Datadog Agent para recopilar métricas internas del balanceador de forma más eficiente.

Logging estructurado

HAProxy fue configurado para emitir **logs estructurados (JSON/syslog)**, almacenados localmente y posteriormente recolectados por Datadog.

Esto permite rastrear eventos como:

- Solicitudes HTTP.
- Códigos de estado.
- Tiempo de respuesta.
- Backend seleccionado.
- Errores de conexión.

El logging estructurado facilita la correlación entre eventos y métricas durante el análisis de incidentes.

La arquitectura contempla un **clúster de tres instancias backend**, implementadas con **Spring Boot**, ejecutándose en puertos independientes:

- Backend Server 1 → puerto **8000**
- Backend Server 2 → puerto **8001**
- Backend Server 3 → puerto **8002**

Cada instancia expone una API REST financiera con operaciones CRUD y endpoints de monitoreo.

Endpoints funcionales

Los servicios incluyen recursos como:

/api/clients
/api/stocks
/api/portfolios

Estos endpoints fueron utilizados por Artillery para simular operaciones realistas sobre el sistema.

Endpoints de observabilidad

Cada backend expone información operacional mediante endpoints como:

/api/health
/api/metrics
/api/system
/api/stats

Estos recursos permiten enriquecer el monitoreo, proporcionando telemetría adicional sobre el estado interno de la aplicación.

Endpoints de fallo controlado

También se implementaron endpoints diseñados para provocar respuestas erróneas deliberadas:

/api/failures/*

El propósito de estas rutas consiste en validar:

- Detección de errores.
- Métricas de disponibilidad.
- Correlación de logs.
- Activación de alertas en Datadog.

4. Capa de observabilidad con Datadog Agent

3. Clúster de backends Spring Boot

La arquitectura incorpora un **Datadog Agent** desplegado como contenedor independiente encargado de recopilar métricas, logs y eventos provenientes del sistema.

El agente recibe información desde múltiples fuentes:

Métricas de HAProxy

Recolectadas mediante:

- Unix Socket compartido.
- Endpoint Prometheus.
- Integración oficial:

haproxy.d/conf.yaml

Estas métricas incluyen:

- Request rate.
- Availability.
- Backend health.
- Connection metrics.
- HTTP error rates.

Logs de HAProxy

Los logs generados por rsyslog son enviados hacia Datadog Log Management para facilitar:

- Búsquedas avanzadas.
- Filtrado temporal.
- Correlación de incidentes.

Métricas de aplicación

El agente también monitorea el estado de los backends Spring Boot, integrando métricas relacionadas con consumo de recursos y comportamiento operacional.

Eventos de infraestructura

La arquitectura permite capturar eventos críticos relacionados con:

- Contenedores detenidos.
- Backends DOWN.
- Reinicios.
- Fallos de red.

Toda esta información converge en un punto central de observabilidad.

5. Plataforma cloud Datadog

En el extremo derecho del diagrama se encuentra la plataforma **Datadog Cloud**, responsable del procesamiento y visualización de telemetría.

Pipeline de ingestión

Datadog recibe:

- **Métricas**
- **Logs**
- **Eventos**

provenientes del agente local.

Posteriormente, los datos son procesados y normalizados para su análisis.

Dashboard personalizado

Se diseñó un dashboard capaz de visualizar indicadores clave en tiempo real:

- **Requests/sec**
- **Latencia**
- **Backends activos**
- **Tasa de error**
- **Métricas temporales (timeline)**

Este dashboard permitió correlacionar directamente el comportamiento del sistema con las pruebas ejecutadas mediante Artillery.

Por ejemplo:

- Durante pruebas de carga alta se observó un incremento abrupto de throughput.
- Durante pruebas de error se evidenció un aumento en códigos HTTP fallidos.
- Al detener un backend se reflejó inmediatamente una disminución de nodos activos.

Sistema de alertas

La arquitectura incluye un mecanismo de alertamiento automatizado.

Se configuraron monitores para escenarios como:

Backend DOWN

Cuando HAProxy detecta indisponibilidad de un backend:

backend status = DOWN

Datadog dispara automáticamente una alerta.

Tasa de error > 5%

Cuando el porcentaje de respuestas fallidas supera el umbral configurado:

error rate > 5%

se genera una notificación automática.

Estas alertas fortalecen la capacidad de respuesta operacional y reducen el tiempo de detección ante incidentes.

6. Persistencia y compartición mediante volúmenes Docker

El sistema utiliza **volúmenes compartidos Docker** para facilitar la comunicación entre componentes.

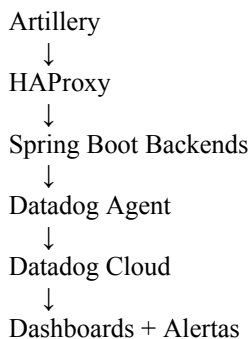
Entre los artefactos compartidos se encuentran:

- **haproxy.cfg**
- Unix socket de HAProxy
- logs de HAProxy
- configuración Datadog Agent

Esto permite desacoplar configuración de infraestructura respecto a los contenedores, favoreciendo mantenibilidad y persistencia.

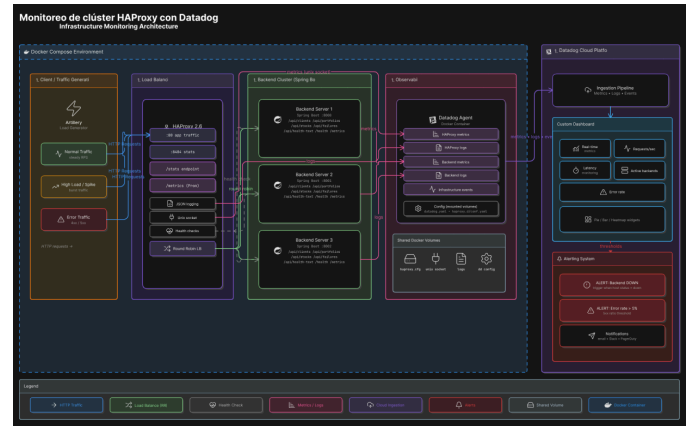
Síntesis del diseño

En términos generales, el diseño implementa un flujo de observabilidad completo:



Esta arquitectura permite no solo balancear carga, sino también observar, medir, diagnosticar y reaccionar frente al comportamiento del sistema en tiempo real, incorporando principios modernos de observabilidad aplicados a infraestructura distribuida.

A) Diagramas de arquitectura



(Ver imagen completa en archivos adjuntos)

V. IMPLEMENTACIÓN

La solución se implementó como una arquitectura distribuida y contenedorizada, compuesta por un balanceador HAProxy, tres instancias de backend Spring Boot, el agente de Datadog para observabilidad, un colector Prometheus para exposición de métricas de HAProxy, y Artillery como generador de carga. El objetivo fue construir un entorno capaz de mostrar tráfico real, registrar logs estructurados, recolectar métricas del balanceador y de los servicios backend, y visualizar todo en Datadog mediante dashboards y alertas.

1. Backends Spring Boot

El núcleo funcional del sistema son tres contenedores backend que ejecutan la misma aplicación Spring Boot, cada uno en un puerto distinto. Para lograrlo, la aplicación se empaquetó con un JAR ejecutable y se configuró el puerto mediante la variable de entorno **SERVER_PORT**, permitiendo levantar tres réplicas idénticas sin modificar el código fuente.

Cada backend expone endpoints orientados a monitoreo y operación, entre ellos rutas de salud, métricas de aplicación y operaciones CRUD para entidades como clientes, acciones y portafolios. Esto permite que el sistema no solo reciba tráfico, sino que también produzca señales útiles para observabilidad.

La ejecución de las tres instancias puede hacerse de forma manual o dentro de Docker con una misma imagen y puertos diferentes, por ejemplo:

```

java -jar app.jar --server.port=8000
java -jar app.jar --server.port=8001
java -jar app.jar --server.port=8002
  
```

En un entorno contenedorizado, esto se traduce en tres servicios backend con la misma imagen y diferentes variables de entorno.

2. HAProxy como balanceador y punto

central de observabilidad

HAProxy se utilizó como balanceador HTTP frontal para distribuir solicitudes entre los tres backends. Además de enrutar tráfico, se configuró para exponer dos capacidades clave: estadísticas internas y métricas Prometheus/OpenMetrics.

La instalación de HAProxy se realizó desde fuente con soporte para Prometheus mediante `USE_PROMEX=1`, lo que habilita el endpoint `/metrics`. También se configuró el backend de estadísticas para consultar el estado del clúster en `/stats`, permitiendo visualizar disponibilidad, sesiones activas, errores y distribución de carga.

La configuración del balanceador incluyó:

- un frontend HTTP en el puerto 80,
- un frontend de estadísticas en el puerto 8404,
- un backend con tres servidores,
- health checks activos,
- y logging estructurado para registrar las solicitudes.

La lógica de balanceo utiliza round-robin, lo que facilita demostrar reparto de carga entre las tres instancias.

3. Logs estructurados en HAProxy

Para cumplir el componente de trazabilidad, se habilitó logging estructurado mediante syslog y formato JSON. HAProxy envía eventos al socket local de syslog, y rsyslog los escribe en un archivo dedicado, por ejemplo `/var/log/haproxy.log`.

El formato JSON permite que cada solicitud quede registrada con campos como:

- timestamp,
- IP del cliente,
- frontend,
- backend,
- servidor destino,
- código HTTP,
- bytes transferidos,
- estado de terminación,
- y tiempos de respuesta.

Esto mejora la capacidad de análisis en Datadog, ya que los logs pueden filtrarse por backend, status code o latencia, y correlacionarse con las métricas del sistema.

4. Datadog Agent e integración con HAProxy

El agente de Datadog se instaló en la máquina host y luego se habilitó la integración de HAProxy mediante el archivo `haproxy.d/conf.yaml`. Con esto, Datadog puede recolectar métricas del balanceador directamente desde el endpoint de estadísticas o desde OpenMetrics, dependiendo de la configuración aplicada.

Además, se activó la recolección de logs para que el archivo generado por HAProxy sea ingerido por Datadog y visualizado en Log Management. Con ello se obtiene una observabilidad más completa: no solo qué pasó, sino también cuándo, dónde y con qué intensidad ocurrió.

La configuración del agente incluyó:

- `datadog.yaml`,
- `haproxy.d/conf.yaml`,
- logs habilitados,
- y la conexión con el sitio `us5.datadoghq.com`.

5. Exposición de métricas con Prometheus/OpenMetrics

Para enriquecer la observabilidad, HAProxy expone su endpoint de métricas en formato Prometheus. Este endpoint permite consultar métricas como:

- conexiones activas,
- número de hilos,
- tiempo de actividad,
- y estadísticas del proceso.

Se configuró un `job` de Prometheus apuntando a `localhost:8404`, de modo que el servicio pudiera leer estas métricas desde el endpoint de HAProxy. En la práctica, esto sirve como una capa adicional de validación y observabilidad, y también facilita integrar herramientas que consumen el formato Prometheus.

La lógica aquí es simple: HAProxy publica métricas, Datadog las recolecta, y Prometheus puede consultarlas para pruebas o validación del endpoint.

6. Prometheus como verificador del endpoint de HAProxy

Aunque Datadog ya cubre la observabilidad principal, Prometheus se empleó como soporte para confirmar que el endpoint `/metrics` funcionaba correctamente. Esto es útil porque permite comprobar de manera independiente que HAProxy está exponiendo métricas válidas antes de integrarlas al dashboard.

La configuración mínima consiste en apuntar a:


```
job_name: 'haproxy'
static_configs:
- targets: ['localhost:8404']
```

y levantar Prometheus con:

```
./prometheus --config.file=prometheus.yml
```

De esta manera se obtiene una validación adicional del canal de métricas.

7. Artillery como generador de tráfico

Artillery se instaló sobre Node.js 20 para evitar incompatibilidades de versión y se usó como herramienta de carga para reproducir tráfico normal, pico y escenarios de error. Su propósito fue generar presión realista sobre el sistema y permitir observar cómo HAProxy distribuye solicitudes y cómo Datadog refleja ese comportamiento en tiempo real.

Las pruebas se diseñaron con escenarios como:

- salud del sistema,
- operaciones CRUD,
- rutas de error controlado,
- y flujos de carga alta con loops y fases escalonadas.

Esto hizo posible comparar el comportamiento del sistema en condiciones normales y bajo estrés, especialmente en métricas como peticiones por segundo, latencia y tasa de error.

8. Observabilidad en Datadog

Con la integración completa, Datadog se utilizó como consola central de análisis. Allí se construyeron dashboards personalizados con métricas de:

- peticiones por segundo,
- latencia,
- backends activos,
- errores 4xx/5xx,
- y disponibilidad del clúster.

También se configuraron monitores para disparar alertas cuando:

- la tasa de error supera un umbral definido,
- un backend cae a estado DOWN,
- o la latencia sobrepasa el límite esperado.

Esto permite no solo visualizar el comportamiento del sistema, sino también reaccionar ante incidentes de forma automatizada.

9. Organización final de la solución

La implementación quedó estructurada en los siguientes bloques funcionales:

- **Backend Spring Boot:** tres instancias idénticas en puertos diferentes.
- **HAProxy:** balanceo de carga, stats y métricas Prometheus.
- **rsyslog:** recepción y almacenamiento de logs de HAProxy en JSON.
- **Datadog Agent:** recolección de métricas y logs.
- **Prometheus:** validación del endpoint de métricas.
- **Artillery:** generación de tráfico y prueba de resiliencia.

10. Resultado operativo

Con esta arquitectura, el sistema logra observarse desde varios ángulos al mismo tiempo: tráfico entrante, estado de balanceo, comportamiento interno de los backends, y eventos de error. Eso convierte la solución en un entorno de monitoreo completo y demostrable, alineado con los requerimientos del proyecto.

VI. PRUEBAS

Con el propósito de validar el comportamiento operativo del clúster implementado, se diseñó y ejecutó un conjunto de escenarios de carga utilizando **Artillery**, una herramienta especializada en pruebas de rendimiento y generación de tráfico HTTP. Estas pruebas no tuvieron únicamente el objetivo de incrementar artificialmente el volumen de solicitudes sobre el sistema, sino también de reproducir condiciones de operación cercanas a un entorno real, permitiendo observar el comportamiento de **HAProxy**, los **backends HTTP**, y la infraestructura de monitoreo en **Datadog** frente a distintos patrones de uso.

La estrategia experimental fue estructurada sobre una progresión de carga compuesta por tres fases: **warmup**, **steady** y **spike**. La fase de *warmup* permitió estabilizar el sistema antes de someterlo a estrés, evitando falsos positivos derivados de procesos de inicialización como el calentamiento de cachés, asignación de conexiones o creación inicial de hilos. Posteriormente, la fase *steady* mantuvo una presión constante sobre el servicio, facilitando la observación del comportamiento normal de la infraestructura bajo carga sostenida. Finalmente, la fase *spike* introdujo un aumento abrupto de solicitudes, reproduciendo un escenario de demanda inesperada y evaluando la capacidad de respuesta del balanceador y de los servicios backend ante picos de tráfico.

Escenario de monitoreo de endpoints

El primer conjunto de pruebas estuvo enfocado en los **endpoints de observabilidad**, incluyendo rutas como `/api/health`, `/api/metrics`, `/api/system` y `/api/stats`. El propósito principal de este escenario consistió en verificar que los servicios destinados al monitoreo permanecieran disponibles y ofrecieran respuestas rápidas incluso bajo condiciones de carga concurrente.

Desde una perspectiva operativa, estos endpoints representan un componente crítico de la arquitectura, dado que plataformas de observabilidad como Datadog dependen de ellos para recopilar información sobre el estado interno del sistema. Un fallo en estos servicios podría impedir la correcta detección de incidentes, afectando la capacidad de diagnóstico y recuperación.

Durante esta prueba se buscó validar no solo la disponibilidad del servicio, sino también la consistencia estructural de las respuestas JSON, asegurando que los agentes de monitoreo pudieran consumir los datos sin interrupciones. Asimismo, esta fase permitió identificar si el incremento de tráfico afectaba negativamente la latencia de las métricas internas, aspecto importante para sistemas donde el monitoreo en tiempo real es un requisito operacional.

Escenario CRUD de clientes

El segundo escenario se centró en el flujo completo de operaciones sobre la entidad **Client**, implementando una secuencia de operaciones **CRUD (Create, Read, Update, Delete)**. Este escenario tuvo como finalidad representar un patrón transaccional típico de aplicaciones empresariales, donde las solicitudes no son independientes, sino que presentan dependencias entre sí.

El proceso inició con la creación de un cliente mediante un **POST**, seguido de consultas individuales (**GET**) para validar la persistencia de la información, actualizaciones (**PUT**) para comprobar la integridad de los cambios, consultas generales (**GET /clients**) para observar el rendimiento de operaciones de listado, y finalmente la eliminación del recurso (**DELETE**).

La importancia de este escenario radica en que reproduce interacciones más complejas que una simple consulta estática. Desde el punto de vista del monitoreo, estas operaciones generan patrones de comportamiento diversos sobre la base de datos y la lógica del backend, permitiendo observar variaciones en tiempos de respuesta, consumo de recursos y distribución de tráfico entre los nodos balanceados por HAProxy.

Adicionalmente, este escenario permitió evaluar si el balanceador mantenía un comportamiento consistente bajo operaciones mixtas de lectura y escritura, verificando que

ninguna instancia backend se convirtiera en un cuello de botella.

Escenario CRUD de acciones bursátiles (Stocks)

El escenario de **Stocks CRUD** fue diseñado bajo una lógica similar al caso anterior, pero enfocado en una entidad diferente asociada al dominio financiero. La ejecución de operaciones de creación, consulta, actualización y eliminación de activos bursátiles tuvo como objetivo validar la estabilidad del backend frente a transacciones de naturaleza distinta, garantizando que las funcionalidades del sistema respondieran de manera uniforme independientemente del recurso solicitado.

Este escenario fue particularmente útil para observar el comportamiento del balanceador frente a rutas adicionales, permitiendo confirmar que el direccionamiento de solicitudes permanecía estable incluso cuando aumentaba la diversidad de endpoints utilizados. Además, facilitó la observación de métricas relacionadas con throughput y latencia promedio, indicadores esenciales para medir el desempeño de APIs REST bajo demanda concurrente.

Escenario de flujo de portafolios

El flujo de **Portfolio** representó el escenario más cercano a un caso de uso real dentro de la aplicación financiera, ya que involucró dependencias entre múltiples entidades. Para crear un portafolio fue necesario generar previamente un cliente y una acción bursátil, reproduciendo relaciones de negocio presentes en sistemas productivos.

A diferencia de los escenarios CRUD individuales, esta prueba presentó un nivel superior de complejidad transaccional, debido a la dependencia secuencial entre recursos. El sistema fue evaluado no solamente por su capacidad para responder solicitudes independientes, sino por mantener consistencia en cadenas completas de operaciones.

Este escenario permitió analizar cómo el incremento de complejidad influía sobre el rendimiento general del sistema y cómo el balanceador distribuía transacciones dependientes entre instancias backend. Asimismo, facilitó la detección de posibles problemas relacionados con tiempos de espera, errores de concurrencia o degradación de latencia bajo carga combinada.

Desde una perspectiva de observabilidad, esta prueba fue especialmente relevante porque permitió correlacionar picos de actividad con métricas internas registradas por Datadog, ofreciendo una visión más precisa del impacto de cargas transaccionales complejas.

Escenario de fallos controlados

Uno de los componentes más relevantes del experimento fue la inclusión de un conjunto de **fallos controlados**, diseñados deliberadamente para provocar respuestas HTTP no exitosas. Se ejecutaron solicitudes hacia endpoints que devolvían errores específicos como **400 (Bad Request)**, **404 (Not Found)**, **503 (Service Unavailable)** y escenarios de **timeout inducido**.

La inclusión de estos errores tuvo un propósito metodológico fundamental: validar la capacidad del sistema de monitoreo para detectar anomalías y reflejarlas correctamente en dashboards y alertas.

En sistemas reales, la observabilidad no se limita al comportamiento exitoso de las aplicaciones; resulta igualmente importante monitorear las condiciones de falla. Por ello, este escenario permitió verificar si Datadog registraba incrementos en la tasa de error, si las métricas reflejaban degradación de disponibilidad y si las alertas configuradas eran disparadas oportunamente ante umbrales críticos.

El endpoint de timeout resultó especialmente útil, ya que permitió observar el comportamiento de la infraestructura frente a respuestas lentas, una condición frecuentemente asociada a saturación, bloqueos internos o degradación progresiva del sistema.

Escenario de carga alta

Finalmente, se ejecutó una **prueba intensiva de carga alta**, orientada a incrementar significativamente el volumen de solicitudes concurrentes sobre endpoints de consulta como `/api/health`, `/api/clients`, `/api/stocks` y `/api/portfolios`.

A través de un mecanismo de repetición (*loop*) con múltiples iteraciones por usuario virtual, se multiplicó artificialmente el número de peticiones procesadas por el sistema. El propósito de esta prueba consistió en someter tanto al balanceador como a los backends a una condición cercana a saturación controlada, evaluando su estabilidad operacional.

Este escenario permitió observar con claridad el comportamiento temporal de métricas críticas en Datadog, incluyendo:

- **Requests per second (RPS):** volumen de solicitudes procesadas por segundo.
- **Latencia promedio:** tiempo de respuesta del sistema durante periodos de alta concurrencia.
- **Estado de backends activos:** disponibilidad de nodos dentro del clúster.
- **Tasa de error:** porcentaje de solicitudes fallidas

frente al total de tráfico.

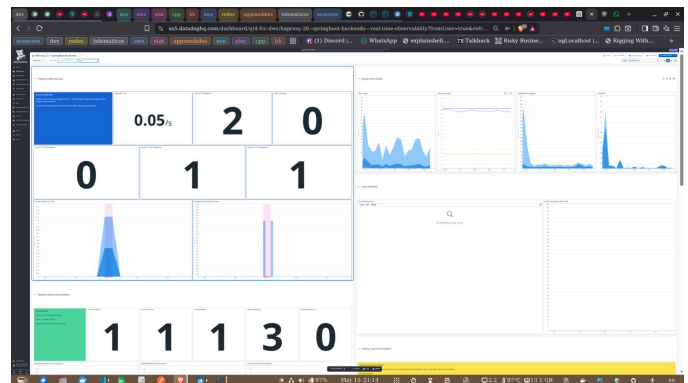
- **Distribución de carga en HAProxy:** equilibrio del tráfico entre instancias.

El análisis comparativo entre los periodos previos, durante y posteriores a la prueba permitió determinar la resiliencia del sistema y verificar la eficacia de HAProxy para absorber incrementos bruscos de demanda sin comprometer significativamente la estabilidad del servicio.

En conjunto, las pruebas realizadas mediante Artillery proporcionaron evidencia empírica del comportamiento del clúster bajo diferentes condiciones de operación, permitiendo correlacionar eventos de carga con métricas observadas en Datadog y fortaleciendo la capacidad de análisis sobre disponibilidad, desempeño y tolerancia a fallos del sistema implementado.

VII. DISCUSIÓN DE LAS PRUEBAS

La fase de pruebas tuvo como propósito validar el correcto funcionamiento de la arquitectura de monitoreo implementada, verificando tanto el comportamiento operativo del clúster HAProxy como la capacidad de observabilidad proporcionada por Datadog frente a distintos escenarios de carga y disponibilidad. En términos generales, los resultados obtenidos fueron consistentes con el comportamiento esperado del sistema, evidenciando una correcta integración entre los componentes desplegados.



Uno de los aspectos más relevantes observados durante las pruebas fue la correspondencia entre los escenarios ejecutados mediante Artillery y las métricas reflejadas en Datadog. A medida que se incrementaba el volumen de solicitudes en las fases de carga (warmup, steady y spike), los dashboards mostraban variaciones proporcionales en indicadores como **peticiones por segundo (requests/sec)**, latencia, throughput y número de conexiones activas. Durante los escenarios de tráfico estable, el comportamiento del sistema se mantuvo consistente, reflejando curvas relativamente uniformes en el dashboard. Por otra parte, durante los escenarios de carga alta, se evidenciaron picos claramente identificables en las gráficas temporales, demostrando que la infraestructura respondía

adecuadamente al incremento de solicitudes concurrentes.

```

[~/Desktop/telematicosdatadogspringboot/telematicosartillery]
$ ./serrano artillery run --scenario-name "Ejecutar prueba Artillery de carga alta" financeinfo-scenarios.yml
Test run id: tpfh_werkefyg49p5mezdbk99yxg6gm_h4r9
Phase started: warmup (index: 0, duration: 30s) 11:30:05(-0500)

-----
Metrics for period to: 11:30:10(-0500) (width: 3.337s)
-----

http.codes.494: ..... 320
http.downloaded.bytes: ..... 1639280
http.request.rate: ..... 99/sec
http.requests: ..... 320
http.response.time: .....
  min: ..... 1
  max: ..... 32
  mean: ..... 4.6
  median: ..... 4
  p95: ..... 10.1
  p99: ..... 16
http.response.time.4xx: .....
  min: ..... 1
  max: ..... 32
  mean: ..... 4.6
  median: ..... 4
  p95: ..... 10.1
  p99: ..... 16
http.responses: ..... 320
users.completed: ..... 8
users.created: ..... 8
users.created.by.name.Ejecutar prueba Artillery de carga alta: ..... 8
users.failed: ..... 0
users.session.length: .....
  min: ..... 215.5
  max: ..... 484.5
  mean: ..... 289.8
  median: ..... 252.2
  p95: ..... 399.5
  p99: ..... 399.5

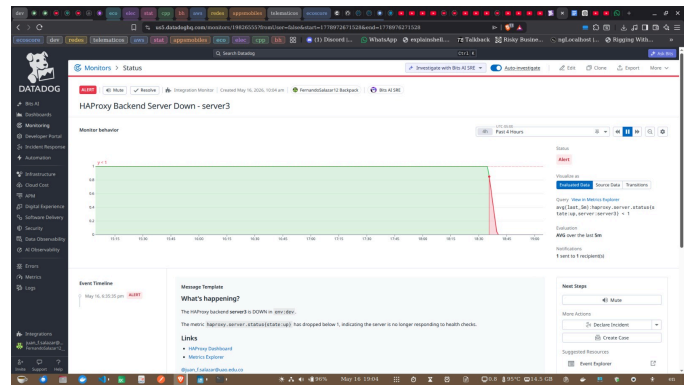
Phase completed: warmup (index: 0, duration: 30s) 11:30:35(-0500)
Phase started: steady (index: 1, duration: 60s) 11:30:35(-0500)

```

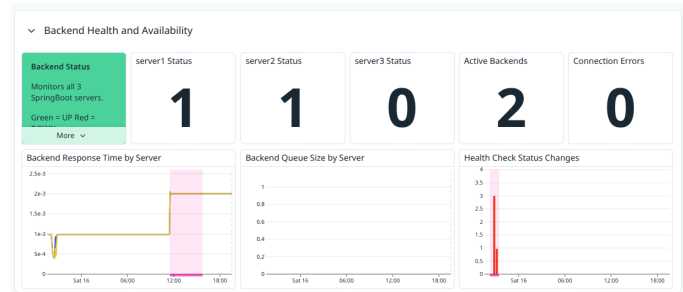
Los escenarios relacionados con operaciones CRUD sobre entidades como clientes, acciones bursátiles (stocks) y portafolios permitieron verificar que las métricas registradas eran coherentes con el patrón de tráfico generado. Las operaciones de creación, consulta, actualización y eliminación produjeron actividad observable tanto en el balanceador como en los backends, generando registros consistentes dentro de Datadog. Esto confirmó que el sistema no solo monitoreaba disponibilidad, sino también comportamiento funcional de la aplicación bajo interacción continua.

En relación con los **endpoints de monitoreo** (`/api/health`, `/api/metrics`, `/api/system` y `/api/stats`), las pruebas evidenciaron tiempos de respuesta rápidos y comportamiento estable. Las métricas reportadas por HAProxy y recolectadas por el Datadog Agent se mantuvieron alineadas con el tráfico generado, lo cual permitió validar la correcta instrumentación de la solución y el adecuado funcionamiento de la integración configurada mediante `haproxy.d/conf.yml`.

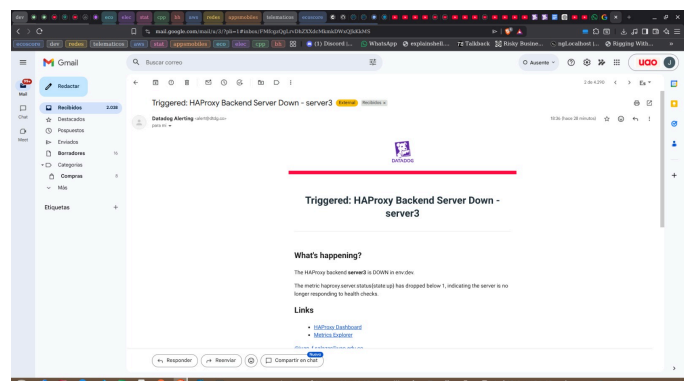
Los escenarios de **fallos controlados** también se comportaron conforme a lo esperado. La ejecución de endpoints que generaban respuestas HTTP no exitosas, tales como códigos **400 (Bad Request)**, **404 (Not Found)**, **503 (Service Unavailable)** y respuestas lentas mediante timeouts, produjo un incremento observable en las tasas de error reflejadas en los dashboards de Datadog. Este comportamiento confirmó que el sistema era capaz de identificar anomalías operacionales y reflejarlas adecuadamente en las métricas de error.



Un aspecto particularmente importante fue la validación del mecanismo de **health checks de HAProxy**. Durante las pruebas donde uno de los backends era detenido intencionalmente, el balanceador detectó correctamente la indisponibilidad del servicio y procedió a marcarlo como **DOWN**, excluyéndolo automáticamente del proceso de balanceo. Esta transición de estado se reflejó de manera inmediata en Datadog, permitiendo observar una reducción en el número de backends activos dentro del dashboard.



De igual forma, el sistema de **alertamiento configurado en Datadog** operó de forma satisfactoria. Las notificaciones por correo electrónico asociadas a eventos críticos, particularmente la caída de un backend o el aumento de la tasa de error sobre los umbrales definidos, fueron enviadas oportunamente. Esto permitió confirmar la correcta configuración de monitores y la capacidad de respuesta temprana ante eventos potencialmente disruptivos dentro de la infraestructura.



Respecto a la capa de **logging**, los registros generados por HAProxy fueron recolectados exitosamente mediante rsyslog y enviados hacia Datadog Log Management. Durante las pruebas fue posible visualizar eventos relacionados con solicitudes HTTP, códigos de respuesta, estados de backend y tiempos de ejecución, facilitando la correlación entre tráfico, comportamiento del sistema y métricas observadas. Esta capacidad de trazabilidad resultó útil para comprender la secuencia de eventos durante escenarios de alta carga y fallos simulados.

Finalmente, la comparación entre los estados del sistema **antes, durante y después de las pruebas de carga** permitió evidenciar un comportamiento consistente de la plataforma. Antes de la carga, las métricas se mantenían estables y con bajo consumo; durante los escenarios de alta intensidad se registraron incrementos significativos y esperados en actividad; y una vez finalizadas las pruebas, las métricas retornaron progresivamente a sus valores normales. Este comportamiento confirma la estabilidad del entorno implementado y valida la capacidad del sistema para adaptarse dinámicamente a variaciones en el volumen de tráfico sin comprometer la observabilidad operacional.

VIII. CONCLUSIONES

El desarrollo del proyecto permitió implementar un entorno integral de monitoreo y observabilidad sobre un clúster de servicios HTTP balanceados mediante HAProxy, integrando herramientas modernas de recolección de métricas, centralización de logs y generación de carga controlada. A través de la combinación de HAProxy, Datadog, Prometheus, Docker y Artillery, se logró construir una arquitectura capaz de evidenciar el comportamiento dinámico de los servicios backend frente a distintos niveles de demanda, permitiendo visualizar de forma cuantitativa indicadores críticos de desempeño.

Uno de los hallazgos más relevantes del proyecto fue la importancia de la observabilidad como componente esencial dentro de sistemas distribuidos y balanceados. La implementación del Datadog Agent con integración HAProxy permitió recopilar métricas de infraestructura y aplicación, tales como solicitudes por segundo, disponibilidad de backends, latencia y tasas de error. Esta instrumentación facilitó la detección temprana de anomalías operacionales y evidenció cómo pequeñas variaciones en la carga podían impactar significativamente el rendimiento general del sistema.

La incorporación de escenarios de tráfico mediante Artillery demostró ser un mecanismo efectivo para validar el comportamiento del sistema bajo diferentes patrones de consumo. Las pruebas de carga progresiva, operaciones CRUD, escenarios dependientes de entidades y endpoints de falla controlada permitieron generar una visión amplia del desempeño real de la API. En particular, los escenarios de

errores inducidos mostraron la utilidad de los monitores configurados en Datadog, ya que permitieron verificar la activación de alertas automáticas ante condiciones críticas como respuestas HTTP 5xx, tiempos de espera elevados o indisponibilidad de servicios backend.

Desde el punto de vista arquitectónico, el uso de HAProxy como balanceador de carga confirmó su eficacia para distribuir solicitudes entre múltiples instancias backend, incrementando la tolerancia a fallos y reduciendo puntos únicos de falla. La posibilidad de exponer métricas vía endpoint Prometheus enriqueció el ecosistema de monitoreo, facilitando la interoperabilidad entre herramientas y proporcionando un mecanismo estandarizado para el análisis del comportamiento interno del balanceador.

Asimismo, la contenerización mediante Docker simplificó significativamente el despliegue y la reproducibilidad del entorno, permitiendo encapsular dependencias complejas como HAProxy compilado con soporte Prometheus, servicios Spring Boot, Datadog Agent y herramientas de testing. Este enfoque favorece prácticas DevOps modernas, ya que reduce inconsistencias entre ambientes y agiliza la administración de componentes distribuidos.

Durante la implementación también se evidenciaron desafíos técnicos relevantes, especialmente relacionados con configuraciones de health checks, dependencias de Node.js para Artillery, integración de módulos de observabilidad y ajustes de logging estructurado en HAProxy. Estos inconvenientes permitieron comprender que los sistemas de monitoreo no solo requieren herramientas sofisticadas, sino también una configuración rigurosa y validaciones continuas para asegurar la calidad de los datos recolectados.

En términos generales, el proyecto cumplió satisfactoriamente con los objetivos planteados, logrando implementar un sistema de monitoreo funcional, escalable y orientado a la observabilidad operacional. La experiencia permitió comprender cómo las organizaciones modernas dependen de plataformas de telemetría en tiempo real para anticipar incidentes, optimizar recursos y mantener altos niveles de disponibilidad. Finalmente, el proyecto evidenció que la integración entre balanceadores, sistemas de métricas, dashboards y pruebas de carga constituye una práctica indispensable para la administración eficiente de infraestructuras distribuidas en entornos empresariales contemporáneos.

REFERENCIAS

- [1] [1] Datadog, *HAProxy Integration Documentation*, Datadog Documentation, 2026. [Online]. Available: <https://us5.datadoghq.com/integrations?integrationId=haproxy>. [Accessed: May 15, 2026].
- [2] [2] HAProxy Technologies, “HAProxy Exposes a Prometheus Metrics Endpoint,” *HAProxy Blog*, 2026. [Online]. Available:

- <https://www.haproxy.com/blog/haproxy-exposes-a-prometheus-metrics-endpoint>. [Accessed: May 15, 2026].
- [3] [3] HAProxy Technologies, *HAProxy Source Repository*, GitHub, 2026. [Online]. Available: <https://github.com/haproxy/haproxy.git>. [Accessed: May 15, 2026].
- [4] [4] Datadog, *Datadog Agent Supported Platforms Documentation*, Datadog Documentation, 2026. [Online]. Available: https://docs.datadoghq.com/agent/supported_platforms/linux/. [Accessed: May 15, 2026].
- [5] [5] Datadog, *Datadog Fleet Agent Installation Portal*, Datadog Cloud Platform, 2026. [Online]. Available: <https://us5.datadoghq.com/fleet/install-agent/latest?platform=linux>. [Accessed: May 15, 2026].
- [6] [6] Datadog, *Datadog Fleet Management Dashboard*, Datadog Cloud Platform, 2026. [Online]. Available: <https://us5.datadoghq.com/fleet>. [Accessed: May 15, 2026].
- [7] [7] Prometheus Authors, *Prometheus Monitoring System Documentation*, Prometheus.io, 2026. [Online]. Available: <https://prometheus.io/docs/introduction/overview/>. [Accessed: May 15, 2026].
- [8] [8] Prometheus Authors, *Prometheus Configuration Documentation*, Prometheus.io, 2026. [Online]. Available: <https://prometheus.io/docs/prometheus/latest/configuration/configuration/>. [Accessed: May 15, 2026].
- [9] [9] Rsyslog Project, *Rsyslog Documentation*, Rsyslog Documentation Portal, 2026. [Online]. Available: <https://www.rsyslog.com/doc/>. [Accessed: May 15, 2026].
- [10] [10] Artillery Software Inc., *Artillery Load Testing Toolkit Documentation*, Artillery.io, 2026. [Online]. Available: <https://www.artillery.io/docs>. [Accessed: May 15, 2026].
- [11] [11] Artillery Software Inc., *Artillery npm Package Repository*, npm Registry, 2026. [Online]. Available: <https://www.npmjs.com/package/artillery>. [Accessed: May 15, 2026].
- [12] [12] Oracle Corporation, *Java Development Kit (JDK) Documentation*, Oracle Docs, 2026. [Online]. Available: <https://docs.oracle.com/en/java/>. [Accessed: May 15, 2026].
- [13] [13] VMware, Inc., *Spring Boot Documentation*, Spring Official Documentation, 2026. [Online]. Available: <https://docs.spring.io/spring-boot/documentation.html>. [Accessed: May 15, 2026].
- [14] [14] Docker Inc., *Docker Compose Documentation*, Docker Docs, 2026. [Online]. Available: <https://docs.docker.com/compose/>. [Accessed: May 15, 2026].
- [15] [15] Node.js Foundation, *Node.js Documentation*, Node.js Official Documentation, 2026. [Online]. Available: <https://nodejs.org/en/docs>. [Accessed: May 15, 2026].
- [16] [16] npm, Inc., *npm CLI Documentation*, npm Official Documentation, 2026. [Online]. Available: <https://docs.npmjs.com/>. [Accessed: May 15, 2026].